When a Geneva Strategy Works in Echo but Not in HTTP

Ashikuzzaman

A first look at one question from my Geneva experiments

GitHub: Geneva HTTP Delivery Study

The Question

After reading about Geneva, I wanted to try it myself and see what a small experiment could teach me. Geneva judges each strategy through an application plugin, so a strategy that works for one application is not automatically correct for another [1]. That left me with a question. If a strategy gets through an echo test, does it also deliver a complete file over HTTP? And when it does not, what can the client say about why, without guessing?

I used Geneva's own engine, packet actions, evaluator and two mock censors, and picked two segmentation strategies by hand. I did not run the genetic search or try to reproduce the paper. This was a way to learn the framework, and the question came up while I was working with it.

What I Did

Both strategies split the first data packet of the request, one at byte 22 and the other at byte 26. The request begins with `GET / HTTP/1.1\r\nHost: ultrasurf.example`, so the hostname starts at byte 22. Split22 keeps the keyword whole in the second segment, while split26 cuts it into `ultr` and `asurf`.

The echo test was already in Geneva. In my local checkout I fixed three problems: replies that arrive in several pieces, the scoring of partial successes, and the server not being ready when the client started. I did not change Geneva's packet actions or the mock censors.

For HTTP I wrote a small plugin called `filecheck`. It downloads a fixed 256 KiB file and counts it as delivered only if the status is 200, the received length and the Content-Length both match the expected size, the SHA-256 matches the reference, and no transfer error occurs. When a transfer fails it records what the client saw (a reset, a timeout, an HTTP error, an incomplete response) and does not try to say why. I looked at the censor logs and packet captures separately, after the trials.

Main Result

The study has 60 recorded trials: 24 echo, 24 HTTP comparison and 12 HTTP fault trials.

	Echo		HTTP file	
Condition	Drop	Reset	Drop	Reset
Benign, no strategy	3/3	3/3	3/3	3/3
Forbidden, no strategy	0/3	0/3	0/3	0/3
Forbidden, split22	0/3	**3/3**	0/3	**0/3**
Forbidden, split26	3/3	3/3	3/3	3/3

Successful runs out of three, against the drop-based and reset-based mock censors. Echo needs five exact exchanges. HTTP needs status 200, the complete 256 KiB object and a matching SHA-256.

The result that interested me most is split22 against the reset-based censor. It passed all three echo trials and failed all three HTTP trials. Split26 worked in both, and neither censor logged a keyword match for it. Everything here is local, with three runs per condition, so it says nothing about real censors.

Why Might Split22 Behave Differently?

In the echo captures the censor detected the keyword and sent resets. All 75 resets I inspected had sequence numbers 22 bytes behind server data the client had already received. In the HTTP captures the server waits for the whole request before it answers, so the first reset arrived before any response data. Its sequence number matched the client's latest ACK, its checksum was valid, and the client reported a reset while waiting for headers.

That looks consistent with how TCP validates resets [3], but the captures cannot tell me what the kernel actually decided. Echo and HTTP also differ in framing, timing, payload size and number of exchanges. So I see reset timing as a possible explanation, not a demonstrated cause.

Is HTTP 200 Enough?

I also ran four faults against benign requests, three trials each: a response with one changed byte, a truncated body, an HTTP 503, and headers delayed past the timeout. All 12 were rejected. A check on status alone would have accepted six of them, and status plus length would still have accepted the three corrupted ones. The SHA-256 check rejected all of them. I introduced these faults myself, so this only confirms that my criteria work as intended. It does not show that they are new or better than Geneva's own HTTP evaluation.

Connection to Previous Research

Geneva introduced automated discovery of packet-level evasion strategies using genetic search [1]. GET /out later studied application-layer evasion, including HTTP and DNS [2]. My experiment adds no new evasion strategy and does not replace either one. It asks a smaller question: how should we judge a strategy's success when the application and its correctness requirements change?

I recognize that application-aware evaluation and response validation already appear in prior work. A detailed comparison of my plugin with existing evaluation methods remains future work.

Possible Follow-up Experiments

Three follow-ups came to mind. I do not know whether they have already been done.

Isolate response timing. Hold the request, the response and the segmentation fixed, and vary only when the server starts sending relative to the injected resets. Randomize the trial order and repeat on different kernels.

Test workload dependence. Run the same strategies on a few workloads with correctness criteria fixed in advance, and compare strategy success, censor detection and client-observed failure separately, with benign controls.

Try diagnostic probes. Check whether a matched benign and keyword-bearing probe gives more evidence than one failed probe under the same probe budget, and record unresolved cases instead of assigning a cause.

These are exploratory questions, not claimed contributions.

Limitations and Request for Guidance

Everything ran on one VM and kernel, with two simple mock censors, three repetitions, one HTTP object and a fixed condition order. Echo and HTTP differ in several ways at once, so these experiments cannot isolate a cause. I have not looked at HTTPS, real censorship, behavior over time or adaptive strategy selection. The verification techniques are standard and I am not claiming novelty. OONI also notes that network anomalies can stay inconclusive [4]. My later check of logs, packet fields and checksums used the recorded evidence; I did not rerun the trials.

I would really appreciate your view on whether cross-workload validation and client-visible delivery evidence are worth pursuing, or whether there is a better direction for me in this area. I want to learn how to ask and test questions like these more rigorously.

Environment: Geneva commit `61a4a51`; Ubuntu 24.04 VM; Linux `7.0.0-38-generic`; Docker 29.1.3; Python 3.6.14. Geneva's architecture, genetic search and packet primitives belong to their original authors.

References

[1] K. Bock et al., "Geneva: Evolving Censorship Evasion Strategies," ACM CCS, 2019. https://geneva.cs.umd.edu/papers/geneva_ccs19.pdf

[2] M. Harrity, K. Bock, F. Sell, and D. Levin, "GET /out: Automated Discovery of Application-Layer Censorship Evasion Strategies," USENIX Security, 2022. https://www.usenix.org/conference/usenixsecurity22/presentation/harrity

[3] RFC 5961, Section 3.2. https://www.rfc-editor.org/rfc/rfc5961.html#section-3.2

[4] OONI, "Interpreting OONI Data." https://ooni.org/support/interpreting-ooni-data/